

Merge Sort

归并排序

定义

归并排序 ([merge sort](#)) 是高效的基于比较的稳定排序算法。

性质

归并排序基于分治思想将数组分段排序后合并，时间复杂度在最优、最坏与平均情况下均为 $O(n \log n)$ ，空间复杂度为 $O(n)$ 。

归并排序可以只使用 $O(1)$ 的辅助空间，但为便捷通常使用与原数组等长的辅助数组。

过程

合并

归并排序最核心的部分是合并 (merge) 过程：将两个有序的数组 $a[i]$ 和 $b[j]$ 合并为一个有序数组 $c[k]$ 。

从左往右枚举 $a[i]$ 和 $b[j]$ ，找出最小的值并放入数组 $c[k]$ ；重复上述过程直到 $a[i]$ 和 $b[j]$ 有一个为空时，将另一个数组剩下的元素放入 $c[k]$ 。

为保证排序的稳定性，前段首元素小于或等于后段首元素时 ($a[i] \leq b[j]$) 而非小于时 ($a[i] < b[j]$) 就要作为最小值放入 $c[k]$ 。

实现



C/C++Python



数组实现指针实现

```
1 void merge(const int *a, size_t aLen, const int *b, size_t bLen, int *c) {
2     size_t i = 0, j = 0, k = 0;
3     while (i < aLen && j < bLen) {
4         if (b[j] < a[i]) { // <!=> 先判断 b[j] < a[i], 保证稳定性
5             c[k] = b[j];
6             ++j;
7         } else {
8             c[k] = a[i];
9             ++i;
10        }
11        ++k;
12    }
13    // 此时一个数组已空，另一个数组非空，将非空的数组并入 c 中
14    for (; i < aLen; ++i, ++k) c[k] = a[i];
15    for (; j < bLen; ++j, ++k) c[k] = b[j];
16 }
```

```

1 void merge(const int *aBegin, const int *aEnd, const int *bBegin,
2           const int *bEnd, int *c) {
3     while (aBegin != aEnd && bBegin != bEnd) {
4         if (*bBegin < *aBegin) {
5             *c = *bBegin;
6             ++bBegin;
7         } else {
8             *c = *aBegin;
9             ++aBegin;
10        }
11        ++c;
12    }
13    for (; aBegin != aEnd; ++aBegin, ++c) *c = *aBegin;
14    for (; bBegin != bEnd; ++bBegin, ++c) *c = *bBegin;
15 }

```

也可使用 `<LAlgorithm>` 库的 `merge` 函数，用法与上述指针式写法的相同。

```

1 def merge(a, b):
2     i, j = 0, 0
3     c = []
4     while i < len(a) and j < len(b):
5         # <!-- 先判断 b[j] < a[i], 保证稳定性
6         if b[j] < a[i]:
7             c.append(b[j])
8             j += 1
9         else:
10            c.append(a[i])
11            i += 1
12    # 此时一个数组已空，另一个数组非空，将非空的数组并入 c 中
13    c.extend(a[i:])
14    c.extend(b[j:])
15    return c

```

分治法实现归并排序

1. 当数组长度为 1 时，该数组就已经是有序的，不用再分解。
2. 当数组长度大于 1 时，该数组很可能不是有序的。此时将该数组分为两段，再分别检查两个数组是否有序（用第 1 条）。如果有序，则将它们合并为一个有序数组；否则对不有序的数组重复第 2 条，再合并。

用数学归纳法可以证明该流程可以将一个数组转变为有序数组。

为保证排序的复杂度，通常将数组分为尽量等长的两段（ $\frac{n}{2}$ ）。

实现

注意下面的代码所表示的区间分别是 $[l, r)$ ， $[l, r]$ ， (l, r) 。



C/C++Python

```

1 void merge_sort(int *a, int l, int r) {
2     if (r - l <= 1) return;
3     // 分解
4     int mid = l + ((r - l) >> 1);
5     merge_sort(a, l, mid), merge_sort(a, mid, r);
6     // 合并
7     int tmp[1024] = {}; // 请根据实际情况设置 tmp 数组的长度 (与 a 相同), 或使用
8                          // vector; 先将合并的结果放在 tmp 里, 再返回到数组 a
9     merge(a + l, a + mid, a + mid, a + r, tmp + 1); // pointer-style merge
10    for (int i = l; i < r; ++i) a[i] = tmp[i];
11 }

```

```

1 def merge_sort(a, ll, rr):
2     if rr - ll <= 1:
3         return
4     # 分解
5     mid = (rr + ll) // 2
6     merge_sort(a, ll, mid)
7     merge_sort(a, mid, rr)
8     # 合并
9     a[ll:rr] = merge(a[ll:mid], a[mid:rr])

```

倍增法实现归并排序

已知当数组长度为 n 时, 该数组就已经是有序的。

将数组全部切成长度为 $n/2$ 的段。

从左往右依次合并两个长度为 $n/2$ 的有序段, 得到一系列长度为 n 的有序段;

从左往右依次合并两个长度为 n 的有序段, 得到一系列长度为 $2n$ 的有序段;

从左往右依次合并两个长度为 $2n$ 的有序段, 得到一系列长度为 $4n$ 的有序段;

.....

重复上述过程直至数组只剩一个有序段, 该段就是排好序的原数组。

▼ 为什么是 $n/2$ 而不是 $n/4$

数组的长度很可能不是 2^k , 此时在最后就可能出现长度不完整的段, 可能出现最后一个段是独立的情况。

实现



C/C++Python

```

1 void merge_sort(int *a, size_t n) {
2     int tmp[1024] = {}; // 请根据实际情况设置 tmp 数组的长度 (与 a 相同), 或使用
3                           // vector; 先将合并的结果放在 tmp 里, 再返回到数组 a
4     for (size_t seg = 1; seg < n; seg <= 1) {
5         for (size_t left1 = 0; left1 < n - seg;
6             left1 += seg + seg) { // n - seg: 如果最后只有一个段就不用合并
7             size_t right1 = left1 + seg;
8             size_t left2 = right1;
9             size_t right2 = std::min(left2 + seg, n); // <!=> 注意最后一个段的边界
10            merge(a + left1, a + right1, a + left2, a + right2,
11                tmp + left1); // pointer-style merge
12            for (size_t i = left1; i < right2; ++i) a[i] = tmp[i];
13        }
14    }
15 }

```

```

1 def merge_sort(a):
2     seg = 1
3     while seg < len(a):
4         for l1 in range(0, len(a) - seg, seg + seg):
5             r1 = l1 + seg
6             l2 = r1
7             r2 = l2 + seg
8             a[l1:r2] = merge(a[l1:r1], a[l2:r2])
9     seg <= 1

```

逆序对

相关阅读和参考实现：[逆序对](#)

逆序对是 且 的有序数对。

排序后的数组无逆序对。归并排序的合并操作中，每次后段首元素被作为当前最小值取出时，前段剩余元素个数之和即是合并操作减少的逆序对数量；故归并排序计算逆序对数量的时间复杂度为 $O(n \log n)$ 。此外，逆序对计数还可以通过树状数组或线段树解决，时间复杂度也是 $O(n \log n)$ ；这一算法的详细解释参见[树状数组](#)相应描述。两种算法的参考实现都在[逆序对](#)章节。

外部链接

- [Merge Sort - GeeksforGeeks](#)
- [归并排序 - 维基百科，自由的百科全书](#)
- [逆序对 - 维基百科，自由的百科全书](#)

本页面最近更新：2024/10/22 14:27:07，[更新历史](#)

发现错误？想一起完善？[在 GitHub 上编辑此页！](#)

本页面贡献者：[NachtgeistW](#), [c-forrest](#), [ChungZH](#), [Enter-tainer](#), [Great-designer](#), [H-J-Granger](#), [iamtwz](#), [invalid-email-address](#), [Ir1d](#), [IronSublimate](#), [Junyan721113](#), [ksyx](#), [lychees](#), [mcendu](#), [megakite](#), [Menci](#), [minghu6](#), [ouuan](#), [partychicken](#), [Saisyc](#), [shawlleyw](#), [sshwy](#), [Tiphereth-A](#), [untitledunrevised](#), [weisi](#), [Xeonacid](#), [zexpp5](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用